



SERVIDOR DOMÉSTICO MULTIFUNCIÓN

**Ubuntu Server · Nextcloud · Docker · NAS -
Cockpit · Code-Server · MariaDB · PostgreSQL**

Autor: Jaime Llastarry Jansana

Fecha: septiembre 2026

Versión: 1.0



PORTAFOLIO: SERVIDOR DOMÉSTICO MULTIFUNCIÓN	3
1. INTRODUCCIÓN	3
2. ESPECIFICACIONES DEL HARDWARE	3
3. SOFTWARE INSTALADO	4
4. SERVICIOS IMPLEMENTADOS	5
4.1. Nextcloud (Nube Privada)	5
4.2. Samba (NAS)	9
4.3. Cockpit (Panel de Administración)	10
4.4. Code-Server (VS Code en Navegador)	13
4.5. MariaDB (Base de Datos MySQL)	15
4.6. PostgreSQL (Base de Datos Relacional)	16
5. SEGURIDAD IMPLEMENTADA	17
5.1 Firewall (UFW) — Estado de puertos	17
5.2 Certificado HTTPS	18
6. DIAGRAMA DE RED	19
7. COMANDOS CLAVE (REFERENCIA RÁPIDA)	20
8. APRENDIZAJES Y CONCLUSIONES	21
8.1. Conocimientos adquiridos	21
8.2. Habilidades transversales	22
8.3. Conclusión	22
9. AMPLIACIÓN: SERVICIOS DE DESARROLLO, BACKUP Y SEGURIDAD	24
9.1. GESTIÓN DE CONTENEDORES CON COCKPIT DOCKER MANAGER	24
9.2. CODE-SERVER (VS CODE EN EL NAVEGADOR)	24
9.3. BASES DE DATOS (MARIADB Y POSTGRESQL)	25
9.4. SEGURIDAD: HTTPS CON CERTIFICADO AUTOFIRMADO	25
9.5. SISTEMA DE COPIAS DE SEGURIDAD AUTOMÁTICO	26
9.6. ALERTAS POR EMAIL (MSMTP + GMAIL)	26
9.7. AJUSTE DE HORA Y ZONA HORARIA	27
9.8. RESUMEN DE SERVICIOS ACTIVOS	27
9.9. APRENDIZAJES ADQUIRIDOS EN ESTA FASE	28
10. Sistema de Backup Automático	28
10.1. Programación (cron)	29

1. INTRODUCCIÓN

Este portafolio documenta la creación, configuración y puesta en marcha de un **servidor doméstico multifunción**, basado en **Ubuntu Server**. El objetivo principal era construir una infraestructura propia que permitiera:

- Almacenar y sincronizar archivos de forma privada (nube personal).
- Compartir archivos en red local (NAS).
- Disponer de un entorno de desarrollo remoto (VS Code en el navegador).
- Gestionar bases de datos para proyectos propios.
- Administrar el servidor de forma cómoda y segura.

Todo ello **sin depender de servicios de terceros**, garantizando el control total sobre los datos.

2. ESPECIFICACIONES DEL HARDWARE

Componente	Detalle
Equipo	Acer Aspire E5-551
CPU	4 núcleos
RAM	14 GB
Disco interno	256 GB SSD
Disco externo	500 GB (Orico, NTFS)
Red	Ethernet (enp1s0f1)
IP fija	192.168.1.10

3. SOFTWARE INSTALADO

Software	Versión	Función
Ubuntu Server	26.04.1 LTS	Sistema operativo
Docker	28.x	Contenedores
Nextcloud	34.0.3 (Snap)	Nube privada
Cockpit	360-1	Panel de administración web
Code-Server	4.136.2	VS Code en navegador
MariaDB	10.6	Base de datos MySQL
PostgreSQL	15	Base de datos relacional
Samba	4.x	Compartición de archivos en red



Figura 0: Logos del software instalado en el servidor doméstico.

4. SERVICIOS IMPLEMENTADOS

5

4.1. Nextcloud (Nube Privada)

- **URL:** <https://192.168.1.10>
- **Función:** Sincronización de archivos, calendario, contactos.
- **Acceso:** Web, cliente de escritorio (Windows/Ubuntu), app móvil (iPhone).
- **Almacenamiento:** Disco externo Orico.

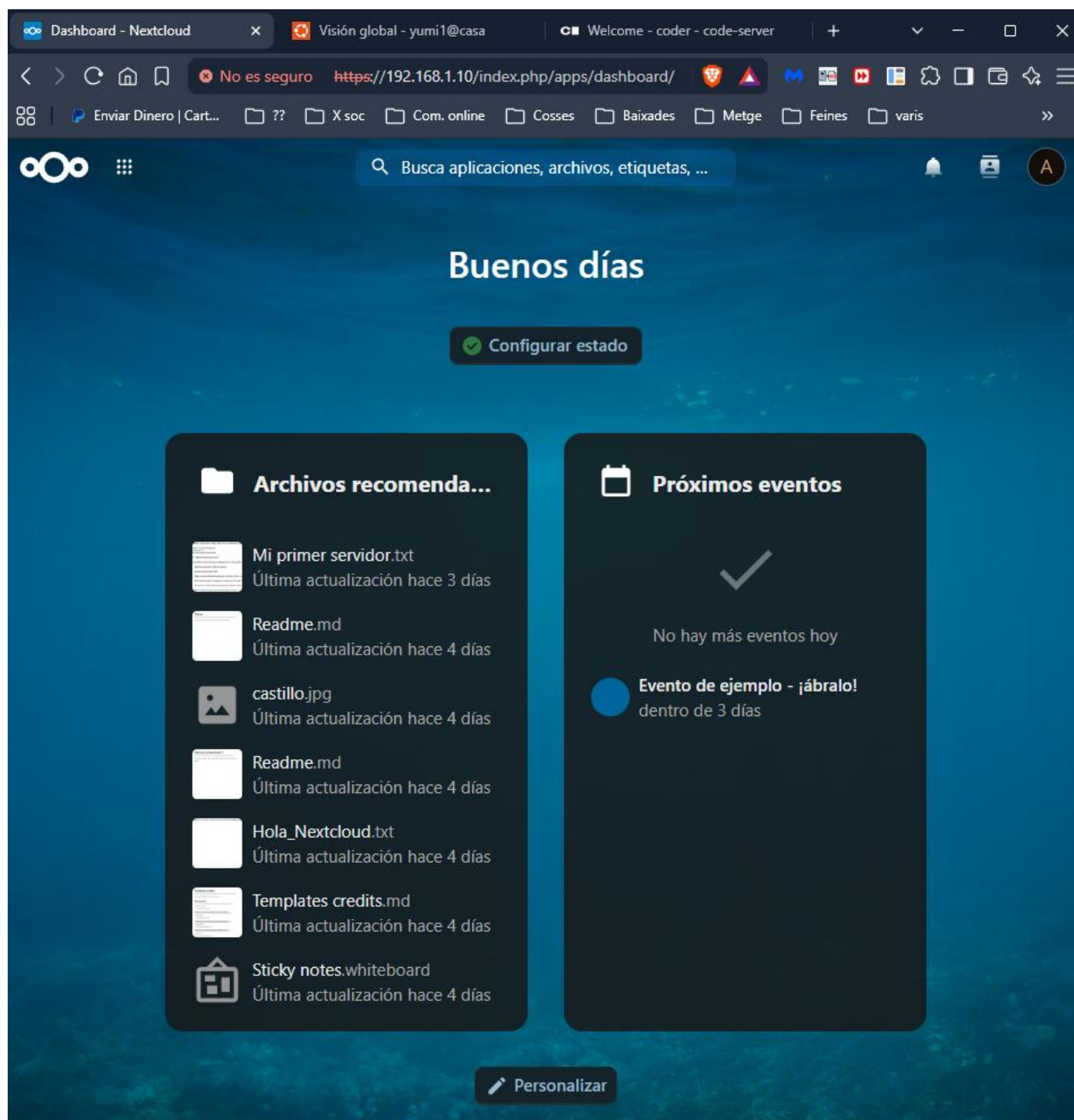


Figura 1: Dashboard de Nextcloud accesible vía HTTPS en <https://192.168.1.10>.

Además del acceso web y de escritorio, Nextcloud se ha configurado para funcionar en dispositivos móviles (iPhone y Android). Esto permite acceder a todos los archivos desde cualquier lugar, subir fotos y vídeos automáticamente, y sincronizar contactos y calendario.

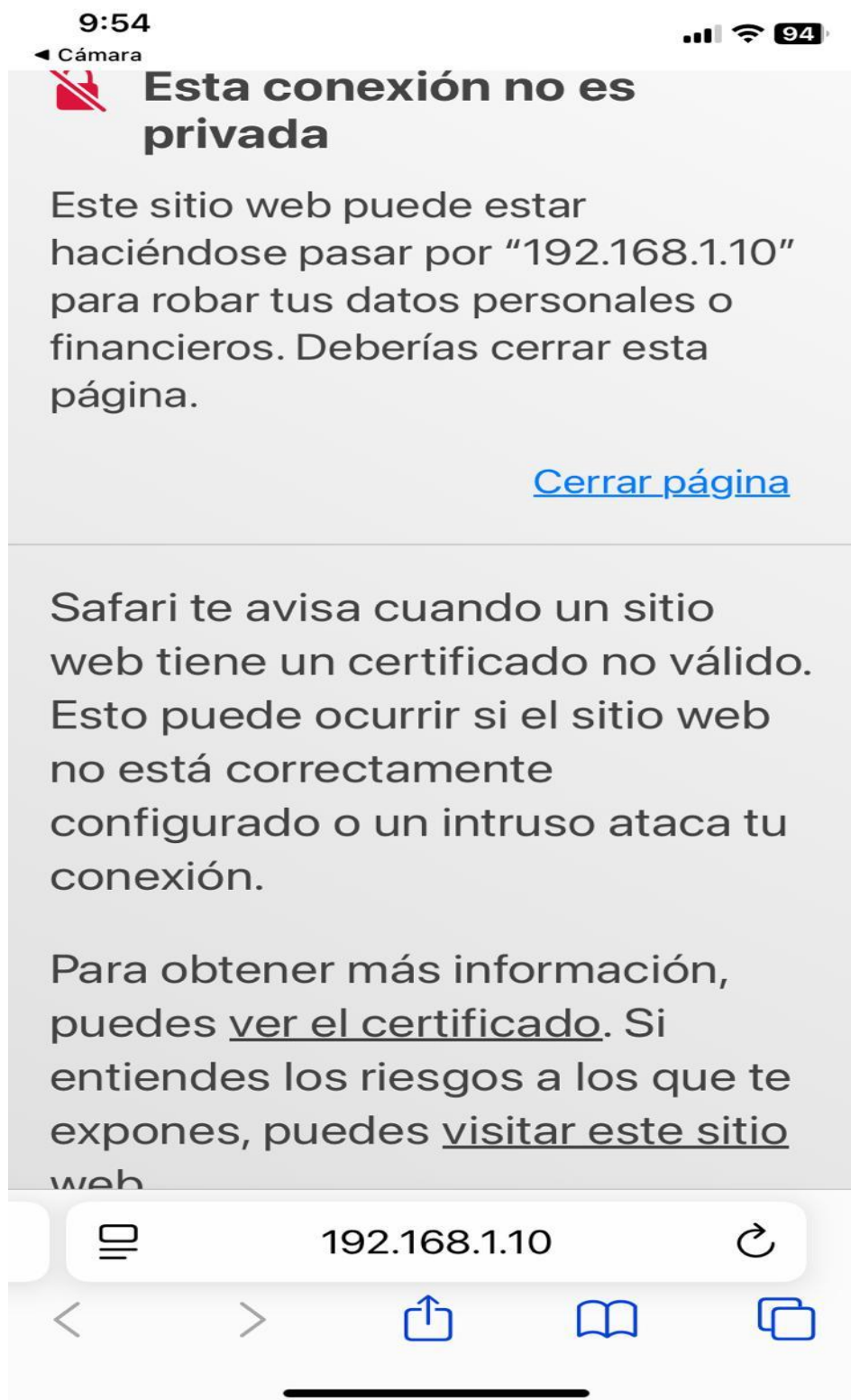


Figura 1.1: Advertencia del certificado auto firmado al acceder a Nextcloud desde Safari en el iPhone.



Figura 1.2: Pantalla de inicio de sesión de la app Nextcloud en el iPhone.

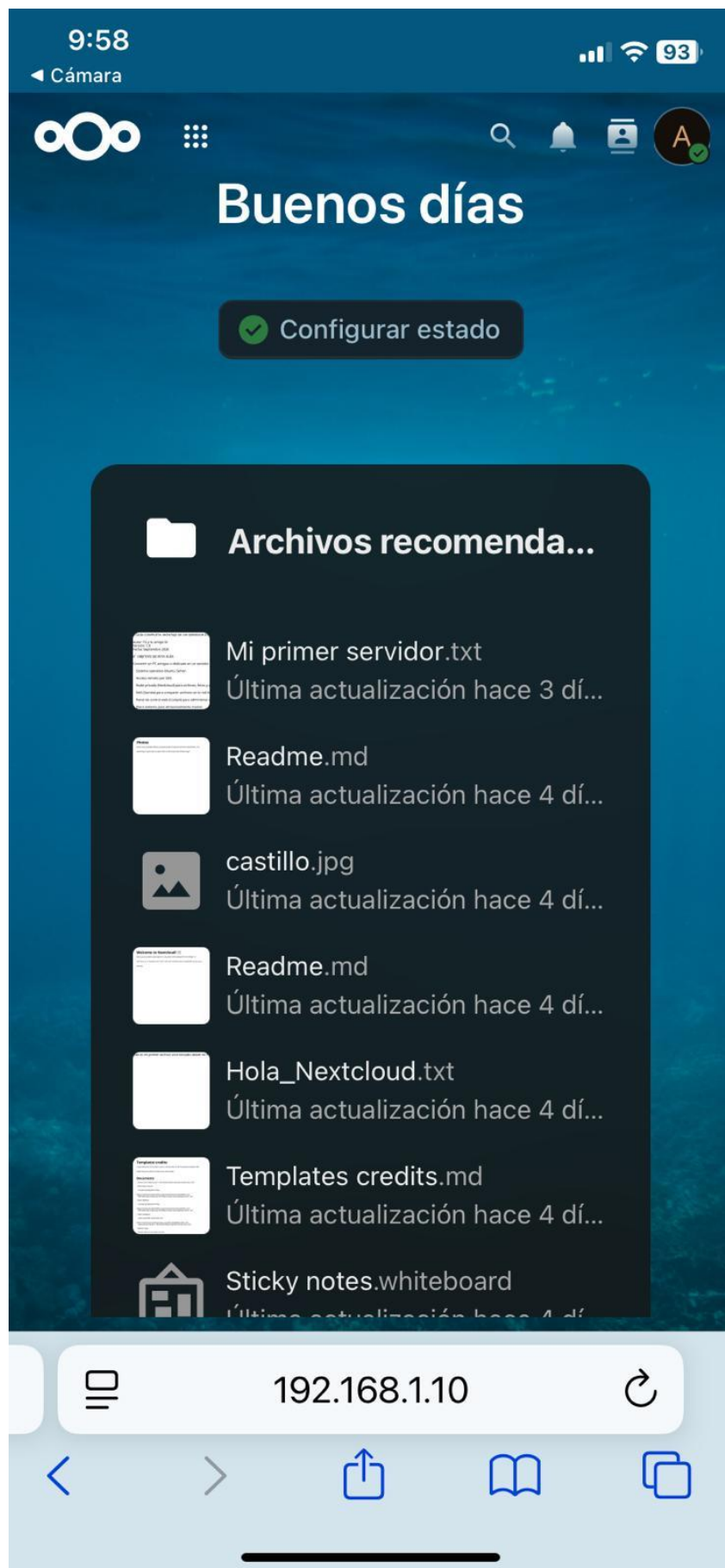


Figura 1.3: Dashboard de Nextcloud accesible desde la app móvil, mostrando los archivos sincronizados.

4.2. Samba (NAS)

- **Ruta:** smb://192.168.1.10/Orico
- **Función:** Compartir el disco externo en la red local.
- **Acceso:** Windows, Linux, macOS.

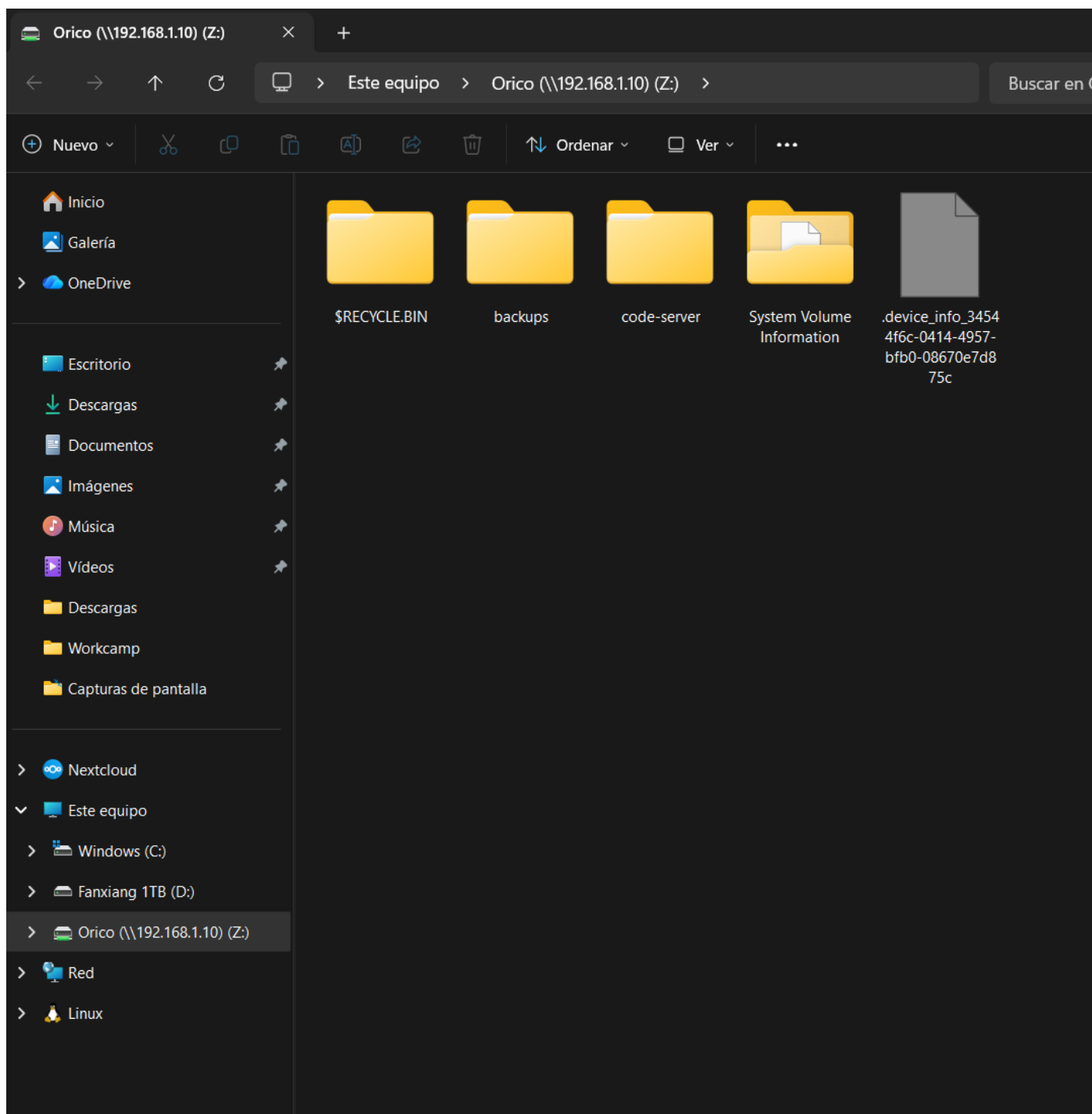


Figura 2: Acceso al disco Orico compartido vía Samba desde Windows, montado como unidad de red Z.

4.3. Cockpit (Panel de Administración)

10

- **URL:** `http://192.168.1.10:9091`
- **Función:** Monitorización del sistema, gestión de servicios, terminal web.
- **Extra:** Cockpit Docker Manager para gestionar contenedores.

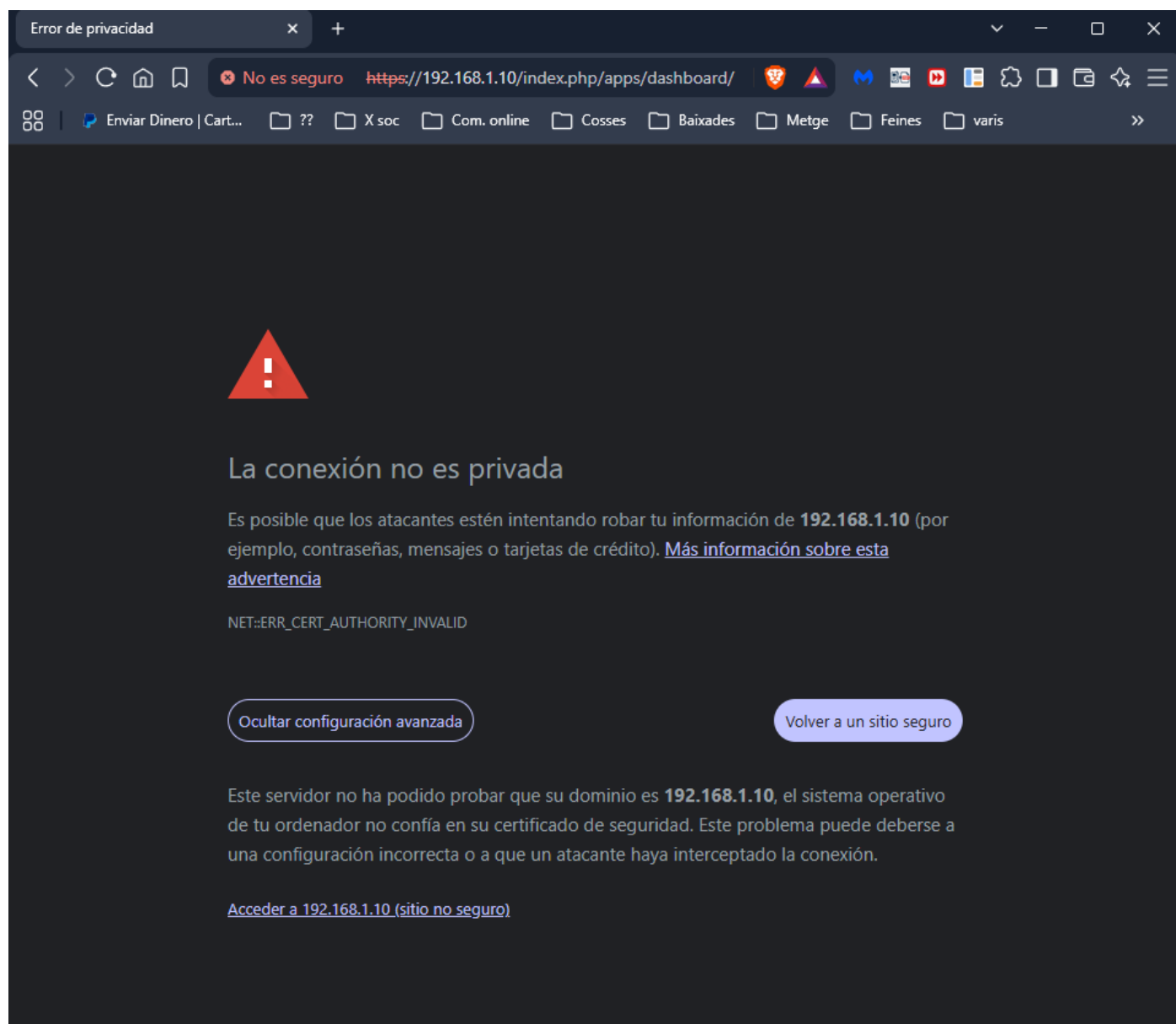


Figura 3: Acceso a Cockpit mediante la URL `http://192.168.1.10:9091`. La advertencia de privacidad se debe al certificado auto firmado utilizado para HTTPS en otros servicios

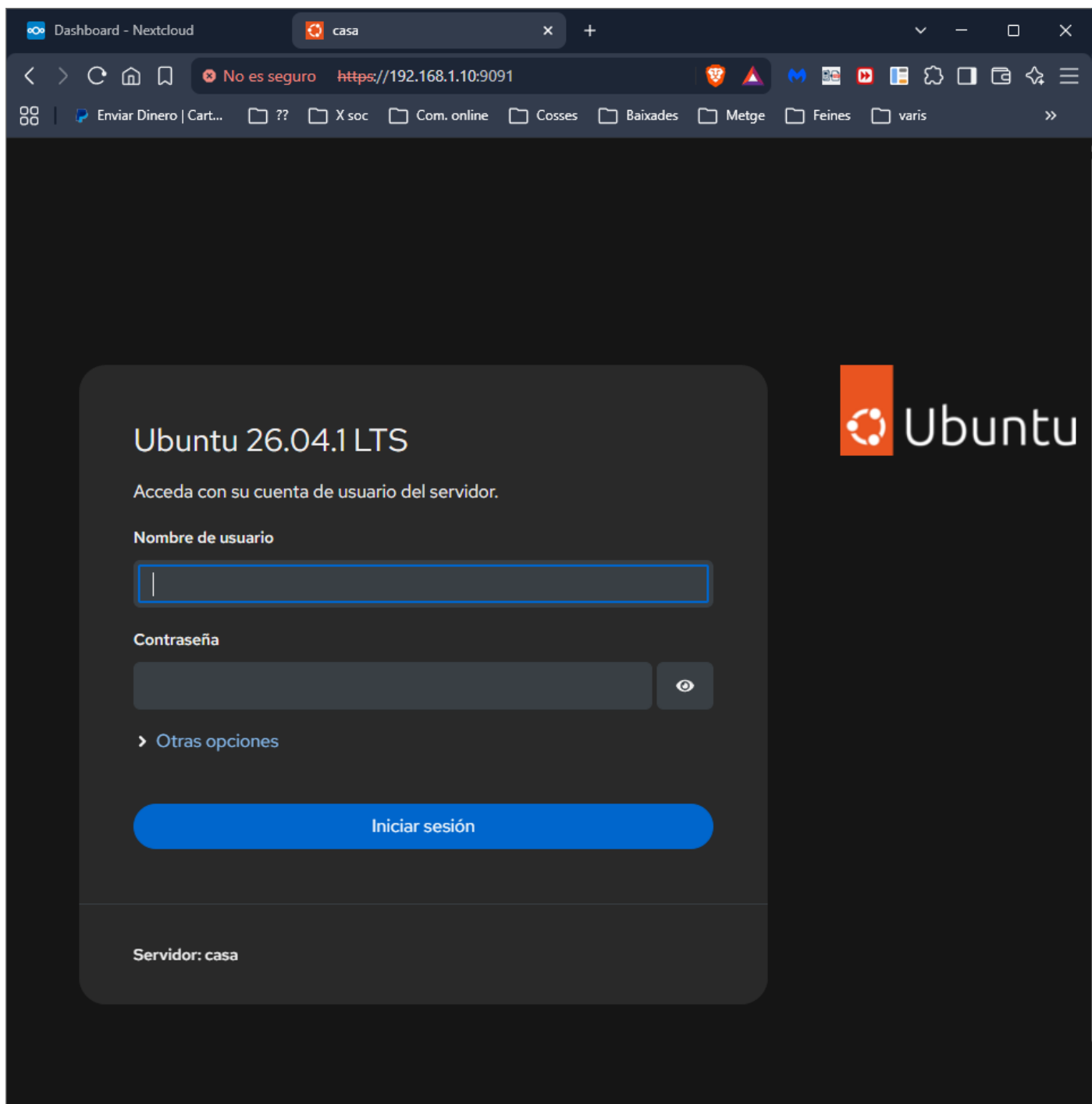


Figura 4: Pantalla de inicio de sesión de Cockpit.

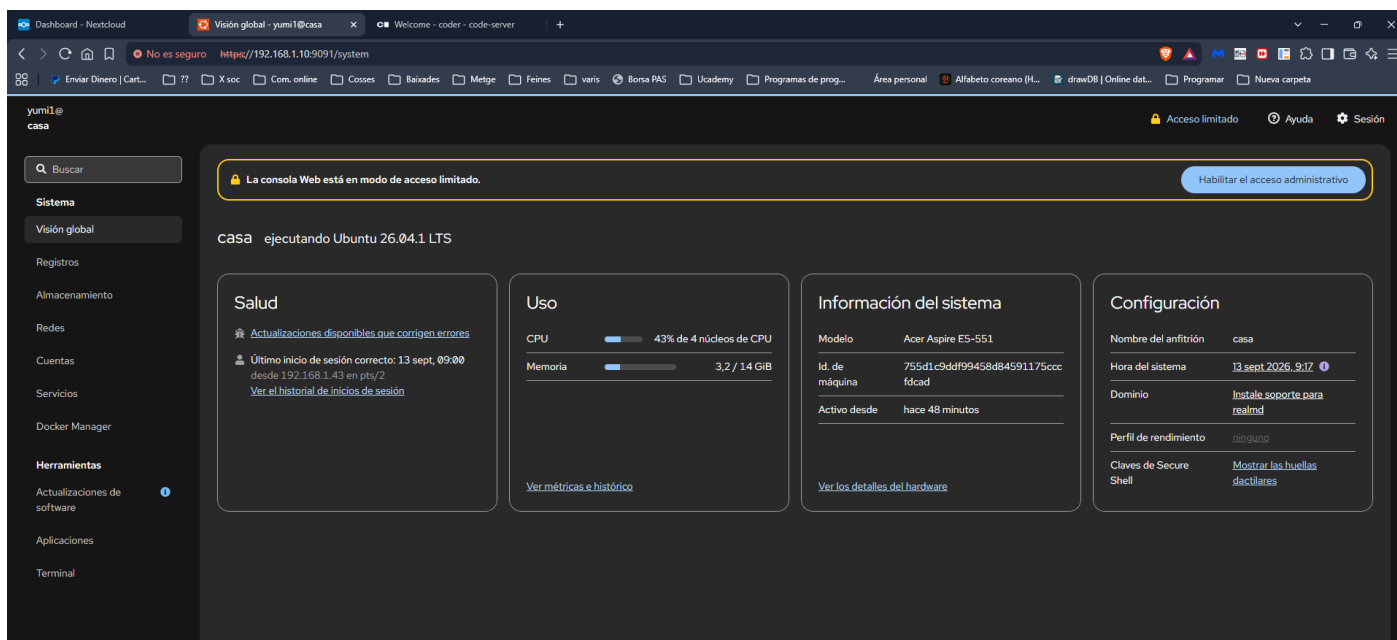


Figura 5: Panel de control de Cockpit mostrando el estado del servidor en tiempo real.

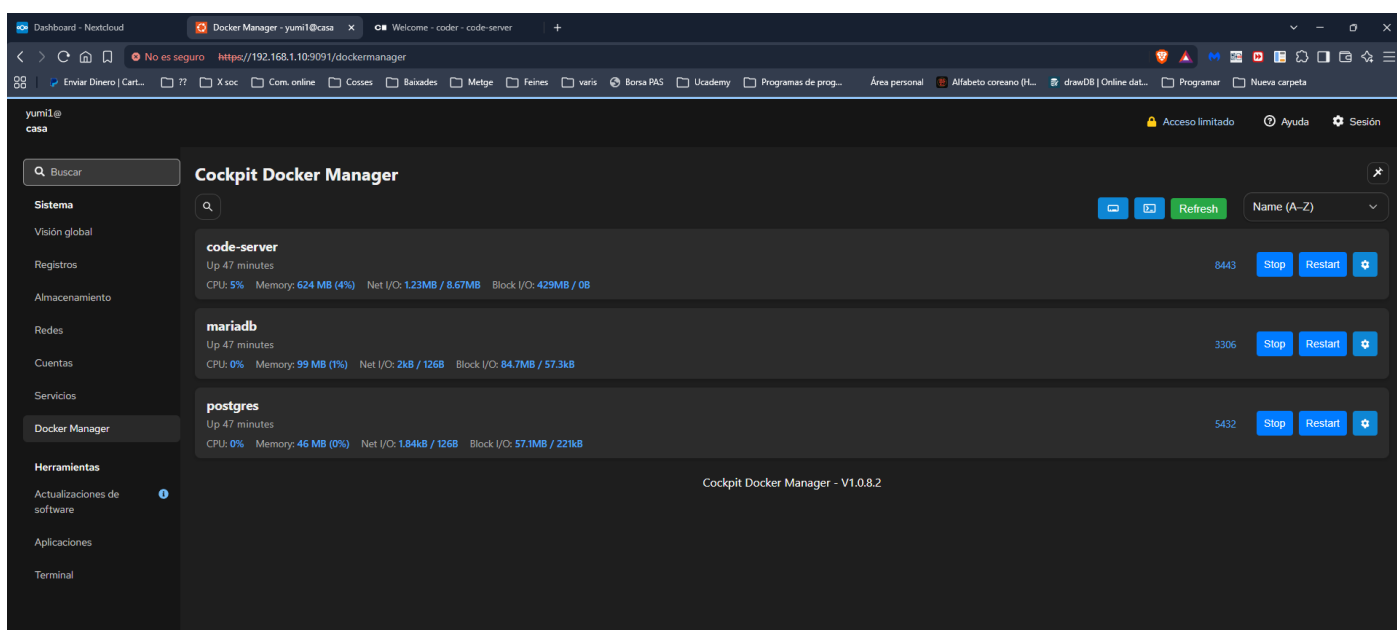


Figura 6: Cockpit Docker Manager, gestión gráfica de contenedores.

Nota: Cockpit permite gestionar el servidor desde el navegador sin necesidad de usar la terminal SSH para tareas comunes (monitoreización, servicios, logs, etc.).

4.4. Code-Server (VS Code en Navegador)

- **URL:** <http://192.168.1.10:8443>
- **Función:** Entorno de desarrollo completo en el navegador.
- **Ventaja:** Programar sin cargar el PC local.
- **Acceso:** Navegador web (Windows, Ubuntu, móvil/Tablet).

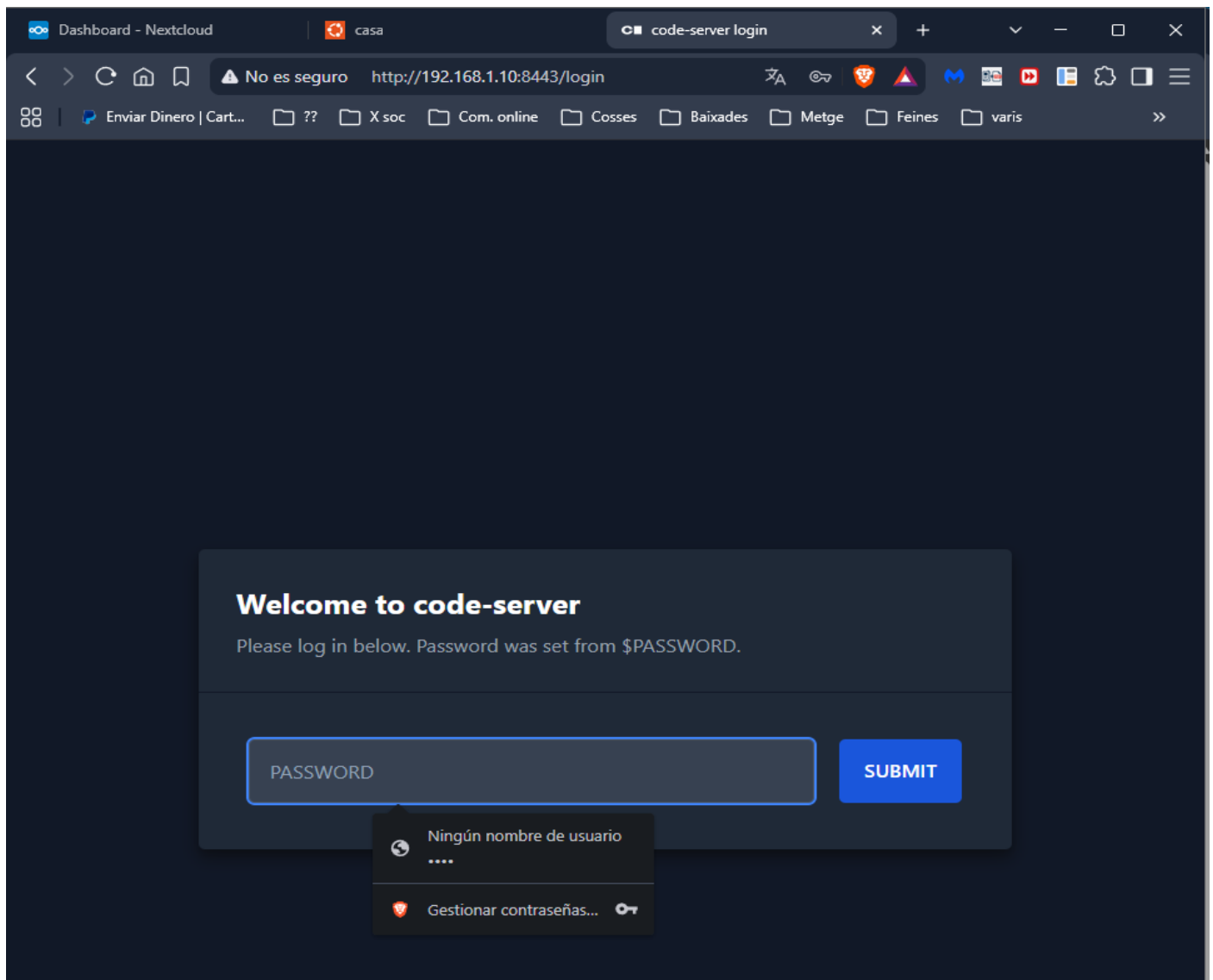


Figura 7: Acceso a Code-server...

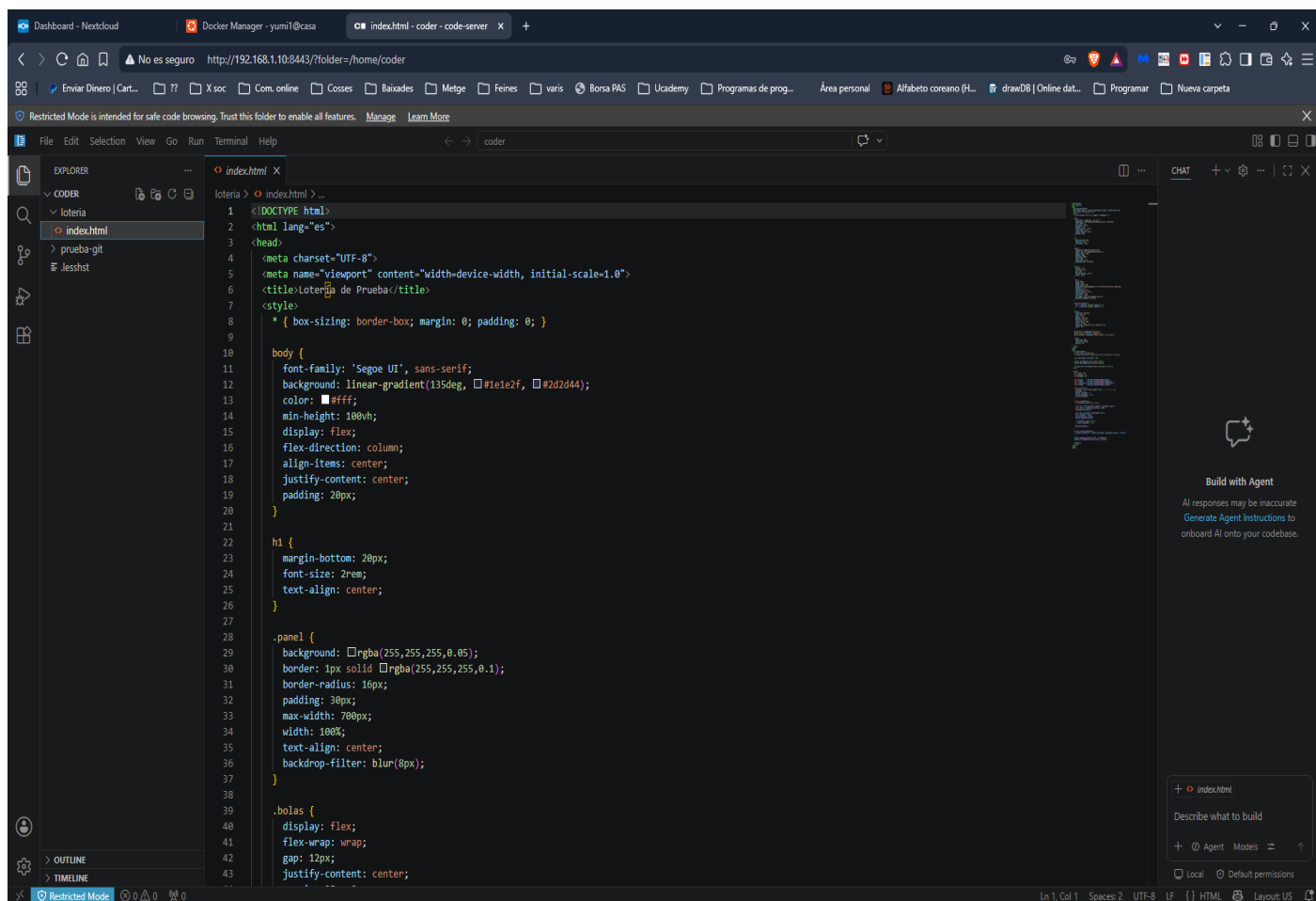


Figura 7.1: Code-Server (interfaz de VS Code)

MariaDB es un sistema de gestión de bases de datos relacional, de código abierto y compatible con MySQL. Se ha instalado en un contenedor Docker para facilitar su gestión y aislamiento, y se ha configurado para almacenar sus datos en el disco interno del servidor.

- Puerto: 3306
- Función: Base de datos para proyectos MySQL.
- Acceso: Cliente DBeaver o terminal (SSH).
- Almacenamiento: /var/lib/mariadb (disco interno).
- Versión: MariaDB 10.6.28.

```
yumi1@casa: ~  
yumi1@casa:~$ docker exec -it mariadb mysql -u root -p  
Enter password:  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 13  
Server version: 10.6.28-MariaDB-ubu2204 mariadb.org binary distribution  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| performance_schema |  
| sys |  
+-----+  
4 rows in set (0.001 sec)  
  
MariaDB [(none)]> SELECT VERSION();  
+-----+  
| VERSION() |  
+-----+  
| 10.6.28-MariaDB-ubu2204 |  
+-----+  
1 row in set (0.000 sec)  
  
MariaDB [(none)]>
```

Figura 8: MariaDB accesible desde la terminal SSH, mostrando las bases de datos disponibles y la versión.

PostgreSQL es un sistema de gestión de bases de datos relacional, orientado a objetos y de código abierto. Se ha instalado en un contenedor Docker para facilitar su gestión y aislamiento, y se ha configurado para almacenar sus datos en el disco interno del servidor.

- Puerto: 5432
- Función: Base de datos para proyectos PostgreSQL.
- Acceso: Cliente DBeaver o terminal (SSH).
- Almacenamiento: /var/lib/postgresql (disco interno).
- Versión: PostgreSQL 15.19

```
yumi1@casa: ~
yumi1@casa:~$ docker exec -it postgres psql -U postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.

postgres=# SELECT version();

              version
-----
PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)

postgres=# \l

               List of databases
   Name   | Owner   | Encoding | Collate | Ctype  | ICU Locale | Locale Provider | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----
postgres | postgres | UTF8     | en_US.utf8 | en_US.utf8 |              | libc              |
template0 | postgres | UTF8     | en_US.utf8 | en_US.utf8 |              | libc              | =c/postgres      +
           |          |          |          |          |          |          | postgres=CTc/postgres
template1 | postgres | UTF8     | en_US.utf8 | en_US.utf8 |              | libc              | =c/postgres      +
           |          |          |          |          |          |          | postgres=CTc/postgres
(3 rows)

postgres=#
```

Figura 9: PostgreSQL accesible desde la terminal SSH, mostrando la versión y las bases de datos disponibles.

Medida	Descripción
Firewall (UFW)	Solo puertos necesarios abiertos
HTTPS	Certificado auto firmado para Nextcloud
SSH seguro	Root deshabilitado, usuario yumi1 con sudo
IP fija	192.168.1.10 configurada con Netplan
Redirección HTTP → HTTPS	Regla de iptables (puerto 80 → 443)
Autenticación Samba	Usuario yumi1 con contraseña específica
Alertas por email	msmtp + Gmail para fallos de backup

5.1 Firewall (UFW) — Estado de puertos

```
yumi1@casa:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
139,445/tcp ALLOW IN 192.168.1.0/24
Anywhere on vxlan.calico ALLOW IN Anywhere
Anywhere on cali+ ALLOW IN Anywhere
9000/tcp ALLOW IN Anywhere
9090/tcp ALLOW IN Anywhere
9091/tcp ALLOW IN Anywhere
8081/tcp ALLOW IN Anywhere
8082/tcp ALLOW IN Anywhere
8443/tcp ALLOW IN Anywhere
3306/tcp ALLOW IN Anywhere
5432/tcp ALLOW IN Anywhere
yumi1@casa:~$ ALLOW OUT Anywhere (v6) on cali+ ico
```

Figura 10: Estado del firewall UFW mostrando los puertos permitidos.

Se ha configurado UFW para permitir únicamente los puertos necesarios:

22 (SSH), 80 (HTTP), 443 (HTTPS), 445/139 (Samba), 3306 (MariaDB),
5432 (PostgreSQL), 8082 (Nextcloud), 8443 (Code-Server), 9091 (Cockpit).

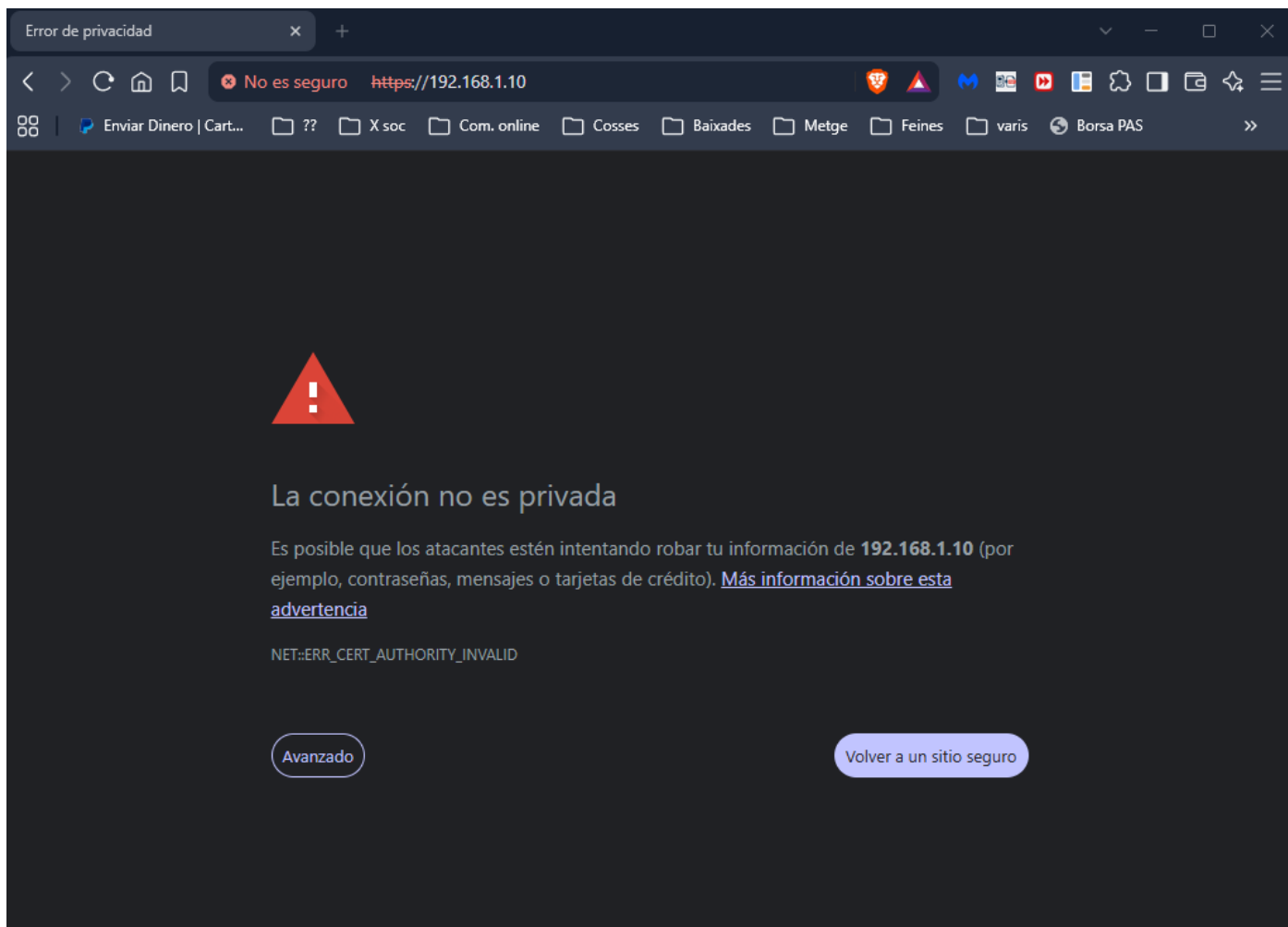


Figura 11: Certificado auto firmado de Nextcloud, mostrando la advertencia del navegador Brave antes de aceptar el certificado.

Se ha generado un certificado auto firmado con OpenSSL para Nextcloud, permitiendo el acceso cifrado mediante HTTPS. Al ser auto firmado, el navegador muestra una advertencia de seguridad la primera vez, que debe aceptarse manualmente.

```
yumi1@casa: ~  
yumi1@casa:~$ sudo cat /etc/ssh/sshd_config | grep -E "PermitRootLogin|AllowUsers|PasswordAuthentication"  
#PermitRootLogin no  
#PasswordAuthentication yes  
# PasswordAuthentication. Depending on your PAM configuration,  
# the setting of "PermitRootLogin prohibit-password".  
# PAM authentication, then enable this but set PasswordAuthentication  
yumi1@casa:~$
```

Figura 12: Configuración de SSH mostrando PermitRootLogin no y PasswordAuthentication yes, reforzando la seguridad del servidor.

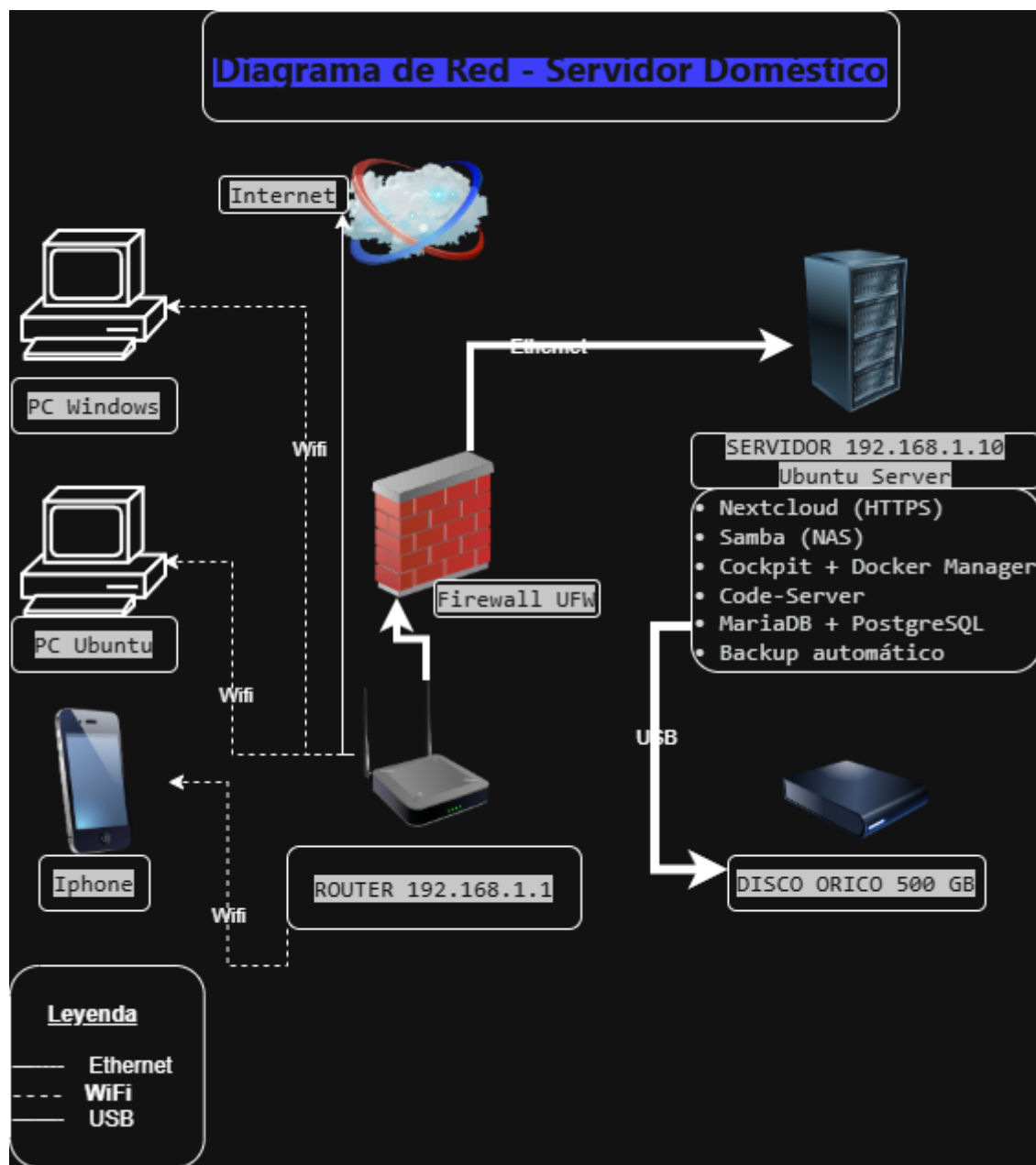


Figura 16: Diagrama de red del servidor doméstico, mostrando los dispositivos conectados, el firewall UFW y los servicios implementados.

7. COMANDOS CLAVE (REFERENCIA RÁPIDA)

A continuación, se presenta una recopilación de los comandos más utilizados en la administración diaria del servidor doméstico. Esta referencia rápida permite gestionar los servicios, consultar estados y ejecutar tareas de mantenimiento sin necesidad de recordar la sintaxis completa:

Comando	Función
<code>ssh yumi1@192.168.1.10</code>	Conectar al servidor
<code>sudo nextcloud.ocp status</code>	Ver estado de Nextcloud
<code>sudo snap restart nextcloud</code>	Reiniciar Nextcloud
<code>sudo systemctl restart smbd</code>	Reiniciar Samba
<code>docker ps</code>	Ver contenedores activos
<code>sudo ufw status verbose</code>	Ver estado del firewall
<code>ip a \ grep "inet "</code>	Ver IP del servidor
<code>sudo smbpasswd -a yumi1</code>	Configurar contraseña de Samba
<code>~/backup.sh</code>	Ejecutar backup manualmente
<code>cat ~/backup.log</code>	Ver log del backup
<code>crontab -l</code>	Ver tareas programadas
<code>docker logs code-server --tail 20</code>	Ver logs de Code-Server
<code>docker logs mariadb --tail 20</code>	Ver logs de MariaDB
<code>docker logs postgres --tail 20</code>	Ver logs de PostgreSQL
<code>sudo timedatectl</code>	Ver hora y zona horaria

```
yumi1@casa:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
0e7df2fcc262   postgres:15    "docker-entrypoint.s..." 33 hours ago   Up 10 hours   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   postgres
2d06190a98ba   mariadb:10.6   "docker-entrypoint.s..." 33 hours ago   Up 10 hours   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   mariadb
512a8aa8617e   codercom/code-server:latest "/usr/bin/entrypoint..." 33 hours ago   Up 10 hours   0.0.0.0:8443->8080/tcp, [::]:8443->8080/tcp   code-server
yumi1@casa:~$
```

Figura 17: Ejemplo de ejecución de comandos en la terminal del servidor.

8.1. Conocimientos adquiridos

Este proyecto me ha permitido adquirir y consolidar conocimientos en múltiples áreas de la informática:

Administración de sistemas:

- Instalación y configuración de Ubuntu Server.
- Gestión de usuarios, permisos y sudo.
- Configuración de red (IP fija con Netplan, DNS, Gateway).
- Administración de servicios con systemd.

Virtualización y contenedores:

- Instalación y gestión de Docker.
- Creación y administración de contenedores.
- Gestión gráfica con Cockpit Docker Manager.

Redes y seguridad:

- Configuración de firewall con UFW.
- Certificados SSL auto firmados con OpenSSL.
- Hardening de SSH (root deshabilitado, usuario autorizado).
- Redirección de puertos con iptables.

Servicios auto alojados:

- Nextcloud (nube privada).
- Samba (NAS en red local).
- Cockpit (panel de administración web).
- Code-Server (VS Code en el navegador).

Bases de datos:

- MariaDB (MySQL).
- PostgreSQL.
- Gestión desde terminal y clientes externos.

Automatización:

- Scripts en Bash para copias de seguridad.
- Programación de tareas con cron.
- Alertas automáticas por email (msmtp + Gmail).
- Limpieza automática de archivos antiguos.

- Resolución de problemas: he aprendido a diagnosticar y solucionar errores de configuración, permisos, redes y servicios.
- Documentación técnica: he elaborado este portafolio como evidencia del trabajo realizado.
- Autonomía: he construido toda la infraestructura sin depender de servicios externos.
- Paciencia y perseverancia: he superado obstáculos como conflictos de puertos, permisos de Samba o certificados SSL.

8.3. Conclusión

He construido una infraestructura completa, segura y funcional, 100% bajo mi control, que me permite:

- Aliviar la carga de mi PC local.
- Disponer de servicios profesionales en casa.
- Aprender y experimentar con tecnologías reales.
- Garantizar la privacidad de mis datos.

Este proyecto no solo me ha enseñado a administrar un servidor, sino también a pensar como un ingeniero de sistemas: planificar, ejecutar, documentar y mejorar.

Lo que empezó como un reto...

Cuando empecé este proyecto, **no sabía si sería capaz de llevar a cabo algo así**. Había oído hablar de servidores, de Docker, de nubes privadas... pero nunca había montado uno desde cero.

Hoy, después de **días de trabajo, errores, soluciones y aprendizaje**, puedo decir con orgullo que tengo un **servidor doméstico completo, seguro y funcional**, con:

- Una **nube privada** (Nextcloud) accesible desde cualquier dispositivo.
- Un **NAS** (Samba) compartiendo archivos en la red local.
- Un **panel de administración web** (Cockpit) para gestionar todo cómodamente.
- Un **entorno de desarrollo remoto** (Code-Server) para programar sin cargar el PC local.
- Dos **bases de datos** (MariaDB y PostgreSQL) listas para proyectos.
- Un **sistema de backup automático** que protege mis datos cada día.
- **Alertas por email** que me avisan si algo falla.

Lo que he aprendido por el camino

Más allá de las tecnologías concretas, este proyecto me ha enseñado **lecciones valiosas**:

- **La importancia de la documentación:** sin apuntar cada paso, cada comando y cada error, habría perdido el hilo en más de una ocasión.
- **La paciencia como herramienta:** muchos problemas no se resuelven a la primera. Hay que probar, investigar y volver a intentarlo.
- **La autonomía:** he aprendido a buscar soluciones por mí mismo, sin depender de que alguien me lo dé todo hecho.
- **La satisfacción del trabajo bien hecho:** ver el servidor funcionando, la nube accesible y los backups ejecutándose solo, es una recompensa que no se paga con dinero.

Mirando al futuro

Este servidor no es un proyecto terminado, sino **una base sobre la que seguir construyendo**. En el futuro, me gustaría:

- **Exponer el servidor a Internet** con un dominio propio y HTTPS válido.
- **Instalar más aplicaciones** en Nextcloud (Maps, Deck, Talk, etc.).
- **Configurar monitorización avanzada** (Netdata, Prometheus).
- **Automatizar copias de seguridad externas** (a otro disco o a la nube).
- **Seguir aprendiendo** sobre administración de sistemas, redes y seguridad.

Reflexión final

Este proyecto me ha demostrado que, **con esfuerzo, constancia y ganas de aprender, se pueden conseguir cosas que parecían imposibles**. No soy un experto en servidores, pero hoy tengo uno funcionando en casa, **bajo mi control, con mis datos y con mis reglas**.

Y eso, para mí, **vale más que cualquier servicio en la nube de pago**.

Tras la puesta en marcha inicial del servidor, se han implementado una serie de mejoras y servicios avanzados que convierten el sistema en una **plataforma completa de desarrollo y gestión de datos**.

9.1. GESTIÓN DE CONTENEDORES CON COCKPIT DOCKER MANAGER

Se ha instalado el módulo **Cockpit Docker Manager**, una interfaz web integrada en Cockpit que permite gestionar contenedores Docker de forma gráfica.

Características:

- Iniciar, detener y reiniciar contenedores.
- Ver logs y estadísticas en tiempo real.
- Gestionar imágenes, volúmenes y redes.
- Crear nuevos contenedores desde la interfaz.

Acceso: <http://192.168.1.10:9091> → Sección "Docker Manager"

9.2. CODE-SERVER (VS CODE EN EL NAVEGADOR)

Se ha instalado **Code-Server**, que permite ejecutar **Visual Studio Code** directamente en el navegador, sin necesidad de instalarlo en el PC local.

Características:

- Editor completo con extensiones.
- Terminal integrada.
- Git integrado.
- Acceso desde cualquier dispositivo de la red.

Acceso: <http://192.168.1.10:8443>

Contraseña: (la configurada en el contenedor)

Ventaja principal: Permite programar usando los recursos del servidor, aliviando la carga del PC local.

9.3. BASES DE DATOS (MARIADB Y POSTGRESQL)

Se han instalado dos sistemas de gestión de bases de datos en contenedores Docker:

Base de Datos	Puerto	Uso
MariaDB	3306	Proyectos MySQL
PostgreSQL	5432	Proyectos PostgreSQL

Configuración:

- Almacenamiento en el disco interno (/var/lib/mariadb y /var/lib/postgresql).
- Acceso remoto desde el PC local con clientes como DBeaver.
- Copias de seguridad automáticas incluidas en el script de backup.

9.4. SEGURIDAD: HTTPS CON CERTIFICADO AUTOFIRMADO

Se ha implementado **HTTPS** en Nextcloud mediante un **certificado auto firmado**, generado con OpenSSL.

Proceso: `sudo nextcloud.enable-https self-signed`

Características:

- Conexión cifrada de extremo a extremo.
- Acceso mediante `https://192.168.1.10`.
- Certificado aceptado manualmente en cada dispositivo (navegador, móvil).

Redirección HTTP → HTTPS:

Se ha configurado una regla de iptables para redirigir el tráfico del puerto 80 al 443:

`sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 443`

9.5. SISTEMA DE COPIAS DE SEGURIDAD AUTOMÁTICO

Se ha implementado un **sistema de backup automático** mediante un script en Bash que se ejecuta diariamente a las 3:00 AM vía cron.

Componentes respaldados:

Componente	Destino	Frecuencia
Nextcloud (archivos)	/mnt/orico/backups/nextcloud/	Diaria
MariaDB	/mnt/orico/backups/bbdd/mariadb_*.sql	Diaria
PostgreSQL	/mnt/orico/backups/bbdd/postgres_*.sql	Diaria
Docker (config)	/mnt/orico/backups/docker/	Diaria
Sistema (/etc)	/mnt/orico/backups/sistema/	Diaria

Script: /home/yumi1/backup.sh

Log: /home/yumi1/backup.log

Programación (cron):

SHELL=/bin/bash

0 3 * * * /bin/bash /home/yumi1/backup.sh >> /home/yumi1/backup.log 2>&1

Características:

- Limpieza automática de backups de más de 30 días.
- Alertas por email si falla algún paso.

9.6. ALERTAS POR EMAIL (MSMTP + GMAIL)

Se ha configurado **msmtp** para enviar alertas por email cuando falla el backup.

Configuración:

- Archivo: ~/.msmtprc
- Proveedor: Gmail (con contraseña de aplicación).
- Función en el script: enviar_alerta "asunto" "mensaje"

Uso:

- El script envía un email automáticamente si falla el backup de MariaDB, PostgreSQL, etc.
- También se puede configurar para recibir un email de confirmación diario (opcional).

Ventaja: Permite saber si algo va mal sin tener que revisar el servidor manualmente.

9.7. AJUSTE DE HORA Y ZONA HORARIA

Se ha configurado la **zona horaria** correcta y la **sincronización automática** con servidores NTP.

Comandos:

```
sudo timedatectl set-timezone Europe/Madrid
```

```
sudo timedatectl set-ntp true
```

Verificación:

```
timedatectl
```

Resultado:

- Hora local: correcta (CEST/CET).
- Sincronización NTP: activa.
- Cron ajustado a la hora local.

9.8. RESUMEN DE SERVICIOS ACTIVOS

Servicio	Acceso	Función
Nextcloud	https://192.168.1.10	Nube privada
Samba (NAS)	smb://192.168.1.10/Orico	Compartición de archivos
Cockpit	http://192.168.1.10:9091	Panel de administración
Cockpit Docker Manager	(desde Cockpit)	Gestión de contenedores
Code-Server	http://192.168.1.10:8443	VS Code en navegador
MariaDB	Puerto 3306	Base de datos MySQL
PostgreSQL	Puerto 5432	Base de datos PostgreSQL
SSH	ssh yumi1@192.168.1.10	Administración remota

- **Gestión de contenedores Docker** con interfaz gráfica (Cockpit).
- **Instalación y configuración de Code-Server** para desarrollo remoto.
- **Despliegue de bases de datos** en contenedores.
- **Configuración de HTTPS** con certificados auto firmados.
- **Automatización de copias de seguridad** con Bash + cron.
- **Envío de alertas por email** con msmtplib.
- **Ajuste de hora y zona horaria** en servidores Linux.

10. Sistema de Backup Automático

```

root@casa: /home/yumi1
root@casa:/home/yumi1# cat /home/yumi1/backup.sh
#!/bin/bash
# Script de copia de seguridad para servidor doméstico
# Autor: yumi1
# Fecha: Septiembre 2026

FECHA=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_DIR="/mnt/orico/backups"

echo "📁 Iniciando copia de seguridad: $FECHA"

# 1. BACKUP DE NEXTCLOUD (archivos)
echo "📁 Copiando archivos de Nextcloud..."
rsync -av --delete /var/snap/nextcloud/common/nextcloud/data/ "$BACKUP_DIR/nextcloud/$FECHA/" > /dev/null 2>&1

# 2. BACKUP DE BASES DE DATOS
echo "📁 Copiando bases de datos..."

# MariaDB
docker exec mariadb sh -c 'exec mysqldump --all-databases -u root -p"$MYSQL_ROOT_PASSWORD"' > "$BACKUP_DIR/bbdd/mariadb_$FECHA.sql" 2>/dev/null
if [ $? -eq 0 ]; then
    echo "✅ MariaDB copiada"
else
    echo "❌ Error en MariaDB"
    enviar_alerta "❌ Error en backup de MariaDB" "El backup de MariaDB ha fallado el $FECHA. Revisa el servidor."
fi

# PostgreSQL
docker exec postgres sh -c 'exec pg_dumpall -U postgres' > "$BACKUP_DIR/bbdd/postgres_$FECHA.sql" 2>/dev/null
if [ $? -eq 0 ]; then
    echo "✅ PostgreSQL copiada"
else
    echo "❌ Error en PostgreSQL"
    enviar_alerta "❌ Error en backup de PostgreSQL" "El backup de PostgreSQL ha fallado el $FECHA. Revisa el servidor."
fi

# 3. BACKUP DE DOCKER (configuración)
echo "📁 Copiando configuración de Docker..."
docker ps --format '{{.Names}}' > "$BACKUP_DIR/docker/contenedores_$FECHA.txt"
docker images --format '{{.Repository}}:{{.Tag}}' > "$BACKUP_DIR/docker/imagenes_$FECHA.txt"

# 4. BACKUP DEL SISTEMA (configuración)
echo "📁 Copiando configuración del sistema..."
tar -czf "$BACKUP_DIR/sistema/etc_$FECHA.tar.gz" /etc 2>/dev/null

# 5. LIMPIAR BACKUPS ANTIGUOS (más de 30 días)
echo "🧹 Limpiando backups antiguos..."
find "$BACKUP_DIR/bbdd" -name "*.sql" -mtime +30 -delete
find "$BACKUP_DIR/nextcloud" -type d -mtime +30 -exec rm -rf {} \; 2>/dev/null
find "$BACKUP_DIR/docker" -name "*.txt" -mtime +30 -delete
find "$BACKUP_DIR/sistema" -name "*.tar.gz" -mtime +30 -delete

# 6. Función para enviar alertas por email
enviar_alerta() {
    local asunto="$1"
    local mensaje="$2"
    echo "$mensaje" | mail -s "$asunto" jaumellastarry@gmail.com
}

# Alerta de éxito (opcional)
#enviar_alerta "✅ Backup completado" "El backup se completó correctamente el $FECHA"

echo "✅ Copia de seguridad completada: $FECHA"
root@casa:/home/yumi1#

```

Figura 13: Script de backup en Bash que automatiza la copia de seguridad de Nextcloud, bases de datos, Docker y configuración del sistema.

```

root@casa:/home/yumi1# cat /home/yumi1/backup.log
[+] Iniciando copia de seguridad: 2026-09-13_10-20-13
[+] Copiando archivos de Nextcloud...
[+] Copiando bases de datos...
    [x] MariaDB copiada
    [x] PostgreSQL copiada
[+] Copiando configuración de Docker...
[+] Copiando configuración del sistema...
[+] Limpiando backups antiguos...
[x] Copia de seguridad completada: 2026-09-13_10-20-13
root@casa:/home/yumi1#

```

Figura 14: Log de ejecución del backup, mostrando la copia correcta de todos los componentes.

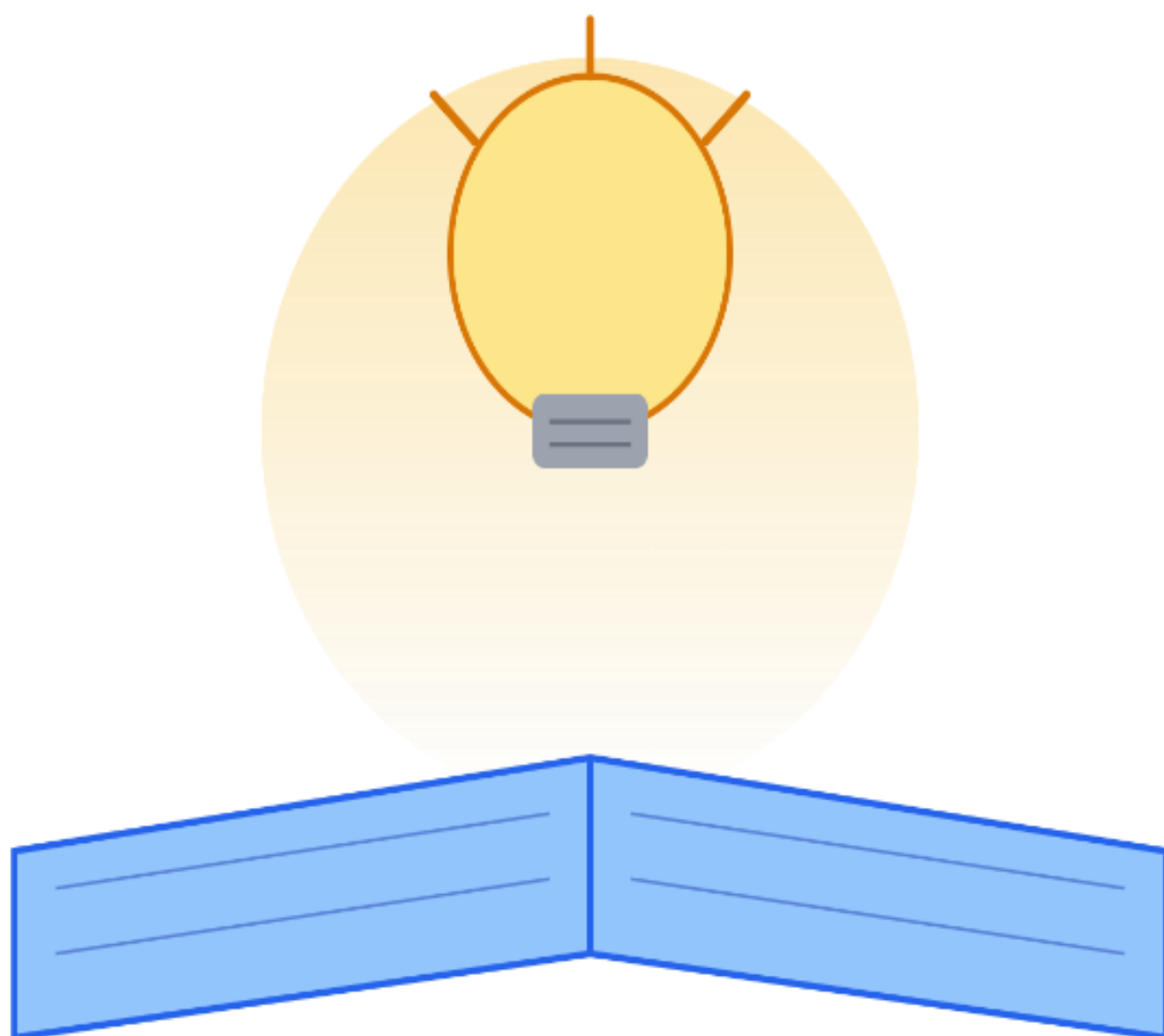
10.1. Programación (cron)

```

root@casa: /home/yumi1
root@casa:/home/yumi1# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
SHELL=/bin/bash
0 3 * * * /bin/bash /home/yumi1/backup.sh >> /home/yumi1/backup.log 2>&1
root@casa:/home/yumi1#

```

Figura 15: Programación del cron para ejecutar el backup automáticamente todos los días a las 3:00 AM.



"El conocimiento es poder,
pero compartirlo es
construir futuro."